Open in Colab

T81-558: Applications of Deep Neural Networks

Module 13: Advanced/Other Topics

- Instructor: Jeff Heaton, McKelvey School of Engineering, Washington University in St. Louis
- For more information visit the class website.

Module 13 Video Material

- Part 13.1: Flask and Deep Learning Web Services [Video] [Notebook]
- **Part 13.2: Interrupting and Continuing Training** [Video] [Notebook]
- Part 13.3: Using a Keras Deep Neural Network with a Web Application [Video] [Notebook]
- Part 13.4: When to Retrain Your Neural Network [Video] [Notebook]
- Part 13.5: Tensor Processing Units (TPUs) [Video] [Notebook]

Google CoLab Instructions

The following code ensures that Google CoLab is running the correct version of TensorFlow.

In [1]:
```
try:
    from google.colab import drive
    COLAB = True
    print("Note: using Google CoLab")
    %tensorflow_version 2.x
except:
    print("Note: not using Google CoLab")
    COLAB = False

# Nicely formatted time string
def hms_string(sec_elapsed):
    h = int(sec_elapsed / (60 * 60))
    m = int((sec_elapsed % (60 * 60)) / 60)
    s = sec_elapsed % 60
    return f"{h}:{m:>02}:{s:>05.2f}"
```

```
Note: using Google CoLab
```

Part 13.2: Interrupting and Continuing Training

We would train our Keras models in one pass in an ideal world, utilizing as much GPU and CPU power as we need. The world in which we train our models is anything but ideal. In this part, we will see that we can stop and continue and even adjust training at later times. We accomplish this continuation with checkpoints. We begin by creating several utility functions. The first utility generates an output directory that has a unique name. This technique allows us to organize multiple runs of our experiment. We provide the Logger class to route output to a log file contained in the output directory.

In [2]:
```
import os
import re
import sys
import time
import numpy as np
from typing import Any, List, Tuple, Union
from tensorflow.keras.datasets import mnist
from tensorflow.keras import backend as K
import tensorflow as tf
import tensorflow.keras
import tensorflow as tf
from tensorflow.keras.callbacks import EarlyStopping, \
  LearningRateScheduler, ModelCheckpoint
from tensorflow.keras import regularizers
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten
from tensorflow.keras.layers import Conv2D, MaxPooling2D
from tensorflow.keras.models import load_model
import pickle

def generate_output_dir(outdir, run_desc):
    prev_run_dirs = []
    if os.path.isdir(outdir):
        prev_run_dirs = [x for x in os.listdir(outdir) if os.path.isdir(\
            os.path.join(outdir, x))]
    prev_run_ids = [re.match(r'^\d+', x) for x in prev_run_dirs]
    prev_run_ids = [int(x.group()) for x in prev_run_ids if x is not None]
    cur_run_id = max(prev_run_ids, default=-1) + 1
    run_dir = os.path.join(outdir, f'{cur_run_id:05d}-{run_desc}')
    assert not os.path.exists(run_dir)
    os.makedirs(run_dir)
    return run_dir

# From StyleGAN2
class Logger(object):
    """Redirect stderr to stdout, optionally print stdout to a file, and
    optionally force flushing on both stdout and the file."""

    def __init__(self, file_name: str = None, file_mode: str = "w", \
```

```
                 should_flush: bool = True):
        self.file = None

        if file_name is not None:
            self.file = open(file_name, file_mode)

        self.should_flush = should_flush
        self.stdout = sys.stdout
        self.stderr = sys.stderr

        sys.stdout = self
        sys.stderr = self

    def __enter__(self) -> "Logger":
        return self

    def __exit__(self, exc_type: Any, exc_value: Any, \
                 traceback: Any) -> None:
        self.close()

    def write(self, text: str) -> None:
        """Write text to stdout (and a file) and optionally flush."""
        if len(text) == 0:
            return

        if self.file is not None:
            self.file.write(text)

        self.stdout.write(text)

        if self.should_flush:
            self.flush()

    def flush(self) -> None:
        """Flush written text to both stdout and a file, if open."""
        if self.file is not None:
            self.file.flush()

        self.stdout.flush()

    def close(self) -> None:
        """Flush, close possible files, and remove
            stdout/stderr mirroring."""
        self.flush()

        # if using multiple loggers, prevent closing in wrong order
        if sys.stdout is self:
            sys.stdout = self.stdout
        if sys.stderr is self:
            sys.stderr = self.stderr

        if self.file is not None:
            self.file.close()

def obtain_data():
    (x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
    print("Shape of x_train: {}".format(x_train.shape))
    print("Shape of y_train: {}".format(y_train.shape))
    print()
    print("Shape of x_test: {}".format(x_test.shape))
    print("Shape of y_test: {}".format(y_test.shape))

    # input image dimensions
    img_rows, img_cols = 28, 28
    if K.image_data_format() == 'channels_first':
        x_train = x_train.reshape(x_train.shape[0], 1, img_rows, img_cols)
        x_test = x_test.reshape(x_test.shape[0], 1, img_rows, img_cols)
        input_shape = (1, img_rows, img_cols)
    else:
        x_train = x_train.reshape(x_train.shape[0], img_rows, img_cols, 1)
        x_test = x_test.reshape(x_test.shape[0], img_rows, img_cols, 1)
        input_shape = (img_rows, img_cols, 1)
    x_train = x_train.astype('float32')
    x_test = x_test.astype('float32')
    x_train /= 255
    x_test /= 255
    print('x_train shape:', x_train.shape)
    print("Training samples: {}".format(x_train.shape[0]))
    print("Test samples: {}".format(x_test.shape[0]))
    # convert class vectors to binary class matrices
    y_train = tf.keras.utils.to_categorical(y_train, num_classes)
    y_test = tf.keras.utils.to_categorical(y_test, num_classes)

    return input_shape, x_train, y_train, x_test, y_test
```

We define the basic training parameters and where we wish to write the output.

In [3]:
```
outdir = "./data/"
run_desc = "test-train"
batch_size = 128
num_classes = 10

run_dir = generate_output_dir(outdir, run_desc)
print(f"Results saved to: {run_dir}")
```

```
Results saved to: ./data/00000-test-train
```

Keras provides a prebuilt checkpoint class named **ModelCheckpoint** that contains most of our desired functionality. This built-in class can save the model's state repeatedly as training progresses. Stopping neural network training is not always a controlled event. Sometimes this stoppage can be abrupt, such as a power failure or a network resource shutting down. If Microsoft Windows is your operating system of choice, your training can also be interrupted by a high-priority system update. Because of all of this uncertainty, it is best to save your model at regular intervals. This process is similar to saving a game at critical checkpoints, so you do not have to start over if something terrible happens to your avatar in the game.

We will create our checkpoint class, named **MyModelCheckpoint**. In addition to saving the model, we also save the state of the training infrastructure. Why save the training

infrastructure, in addition to the weights? This technique eases the transition back into training for the neural network and will be more efficient than a cold start.

Consider if you interrupted your college studies after the first year. Sure, your brain (the neural network) will retain all the knowledge. But how much rework will you have to do? Your transcript at the university is like the training parameters. It ensures you do not have to start over when you come back.

In [4]:
```
class MyModelCheckpoint(ModelCheckpoint):
  def __init__(self, *args, **kwargs):
    super().__init__(*args, **kwargs)

  def on_epoch_end(self, epoch, logs=None):
    super().on_epoch_end(epoch,logs)\

    # Also save the optimizer state
    filepath = self._get_file_path(epoch=epoch,
        logs=logs, batch=None)
    filepath = filepath.rsplit( ".", 1 )[ 0 ]
    filepath += ".pkl"

    with open(filepath, 'wb') as fp:
      pickle.dump(
        {
          'opt': model.optimizer.get_config(),
          'epoch': epoch+1
         # Add additional keys if you need to store more values
        }, fp, protocol=pickle.HIGHEST_PROTOCOL)
    print('\nEpoch %05d: saving optimizaer to %s' % (epoch + 1, filepath))
```

The optimizer applies a step decay schedule during training to decrease the learning rate as training progresses. It is essential to preserve the current epoch that we are on to perform correctly after a training resume.

In [5]:
```
def step_decay_schedule(initial_lr=1e-3, decay_factor=0.75, step_size=10):
    def schedule(epoch):
        return initial_lr * (decay_factor ** np.floor(epoch/step_size))
    return LearningRateScheduler(schedule)
```

We build the model just as we have in previous sessions. However, the training function requires a few extra considerations. We specify the maximum number of epochs; however, we also allow the user to select the starting epoch number for training continuation.

In [6]:
```
def build_model(input_shape, num_classes):
    model = Sequential()
    model.add(Conv2D(32, kernel_size=(3, 3),
                     activation='relu',
                     input_shape=input_shape))
    model.add(Conv2D(64, (3, 3), activation='relu'))
    model.add(MaxPooling2D(pool_size=(2, 2)))
```

```
    model.add(Dropout(0.25))
    model.add(Flatten())
    model.add(Dense(128, activation='relu'))
    model.add(Dropout(0.5))
    model.add(Dense(num_classes, activation='softmax'))
    model.compile(
        loss='categorical_crossentropy',
        optimizer=tf.keras.optimizers.Adam(),
        metrics=['accuracy'])
    return model

def train_model(model, initial_epoch=0, max_epochs=10):
    start_time = time.time()

    checkpoint_cb = MyModelCheckpoint(
        os.path.join(run_dir, 'model-{epoch:02d}-{val_loss:.2f}.hdf5'),
        monitor='val_loss',verbose=1)

    lr_sched_cb = step_decay_schedule(initial_lr=1e-4, decay_factor=0.75, \
                                      step_size=2)
    cb = [checkpoint_cb, lr_sched_cb]

    model.fit(x_train, y_train,
              batch_size=batch_size,
              epochs=max_epochs,
              initial_epoch = initial_epoch,
              verbose=2, callbacks=cb,
              validation_data=(x_test, y_test))
    score = model.evaluate(x_test, y_test, verbose=0, callbacks=cb)
    print('Test loss: {}'.format(score[0]))
    print('Test accuracy: {}'.format(score[1]))

    elapsed_time = time.time() - start_time
    print("Elapsed time: {}".format(hms_string(elapsed_time)))
```

We now begin training, using the **Logger** class to write the output to a log file in the output directory.

In [7]:
```
with Logger(os.path.join(run_dir, 'log.txt')):
    input_shape, x_train, y_train, x_test, y_test = obtain_data()
    model = build_model(input_shape, num_classes)
    train_model(model, max_epochs=3)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mn
ist.npz
11493376/11490434 [==============================] - 0s 0us/step
11501568/11490434 [==============================] - 0s 0us/step
Shape of x_train: (60000, 28, 28)
Shape of y_train: (60000,)

Shape of x_test: (10000, 28, 28)
Shape of y_test: (10000,)
x_train shape: (60000, 28, 28, 1)
Training samples: 60000
Test samples: 10000
Epoch 1/3

Epoch 1: saving model to ./data/00000-test-train/model-01-0.20.hdf5

Epoch 00001: saving optimizaer to ./data/00000-test-train/model-01-0.20.pkl
469/469 - 12s - loss: 0.6354 - accuracy: 0.8129 - val_loss: 0.1977 - val_accuracy:
0.9420 - lr: 1.0000e-04 - 12s/epoch - 25ms/step
Epoch 2/3

Epoch 2: saving model to ./data/00000-test-train/model-02-0.11.hdf5

Epoch 00002: saving optimizaer to ./data/00000-test-train/model-02-0.11.pkl
469/469 - 2s - loss: 0.2284 - accuracy: 0.9332 - val_loss: 0.1087 - val_accuracy: 0.
9677 - lr: 1.0000e-04 - 2s/epoch - 5ms/step
Epoch 3/3

Epoch 3: saving model to ./data/00000-test-train/model-03-0.08.hdf5

Epoch 00003: saving optimizaer to ./data/00000-test-train/model-03-0.08.pkl
469/469 - 2s - loss: 0.1575 - accuracy: 0.9541 - val_loss: 0.0837 - val_accuracy: 0.
9746 - lr: 7.5000e-05 - 2s/epoch - 5ms/step
Test loss: 0.08365701138973236
Test accuracy: 0.9746000170707703
Elapsed time: 0:00:22.09
```

You should notice that the above output displays the name of the hdf5 and pickle (pkl) files produced at each checkpoint. These files serve the following functions:

- Pickle files contain the state of the optimizer.
- HDF5 files contain the saved model.

For this training run, which went for 3 epochs, these two files were named:

- ./data/00013-test-train/model-03-0.08.hdf5
- ./data/00013-test-train/model-03-0.08.pkl

We can inspect the output from the training run. Notice we can see a folder named "00000-test-train". This new folder was the first training run. The program will call the next training run "00001-test-train", and so on. Inside this directory, you will find the pickle and hdf5 files for each checkpoint.

In [13]:
```
!ls ./data
```

```
00000-test-train
```

In [14]:
```
!ls ./data/00000-test-train
```

```
log.txt             model-01-0.20.pkl   model-02-0.11.pkl   model-03-0.08.pkl
model-01-0.20.hdf5  model-02-0.11.hdf5  model-03-0.08.hdf5
```

Keras stores the model itself in an HDF5, which includes the optimizer. Because of this feature, it is not generally necessary to restore the internal state of the optimizer (such as ADAM). However, we include the code to do so. We can obtain the internal state of an optimizer by calling **get_config**, which will return a dictionary similar to the following:

```
{'name': 'Adam', 'learning_rate': 7.5e-05, 'decay': 0.0,
'beta_1': 0.9, 'beta_2': 0.999, 'epsilon': 1e-07, 'amsgrad': False}
```

In practice, I've found that different optimizers implement get_config differently. This function will always return the training hyperparameters. However, it may not always capture the complete internal state of an optimizer beyond the hyperparameters. The exact implementation of get_config can vary per optimizer implementation.

Continuing Training

We are now ready to continue training. You will need the paths to both your HDF5 and PKL files. You can find these paths in the output above. Your values may differ from mine, so perform a copy/paste.

In [15]:
```
MODEL_PATH = './data/00000-test-train/model-03-0.08.hdf5'
OPT_PATH = './data/00000-test-train/model-03-0.08.pkl'
```

The following code loads the HDF5 and PKL files and then recompiles the model based on the PKL file. Depending on the optimizer in use, you might have to recompile the model.

In [16]:
```
import tensorflow as tf
from tensorflow.keras.models import load_model
import pickle

def load_model_data(model_path, opt_path):
    model = load_model(model_path)
    with open(opt_path, 'rb') as fp:
      d = pickle.load(fp)
      epoch = d['epoch']
      opt = d['opt']
      return epoch, model, opt

epoch, model, opt = load_model_data(MODEL_PATH, OPT_PATH)
```

```
# note: often it is not necessary to recompile the model
model.compile(
    loss='categorical_crossentropy',
    optimizer=tf.keras.optimizers.Adam.from_config(opt),
    metrics=['accuracy'])
```

Finally, we train the model for additional epochs. You can see from the output that the new training starts at a higher accuracy than the first training run. Further, the accuracy increases with additional training. Also, you will notice that the epoch number begins at four and not one.

In [17]:
```
outdir = "./data/"
run_desc = "cont-train"
num_classes = 10

run_dir = generate_output_dir(outdir, run_desc)
print(f"Results saved to: {run_dir}")

with Logger(os.path.join(run_dir, 'log.txt')):
  input_shape, x_train, y_train, x_test, y_test = obtain_data()
  train_model(model, initial_epoch=epoch, max_epochs=6)
```

```
Results saved to: ./data/00001-cont-train
Shape of x_train: (60000, 28, 28)
Shape of y_train: (60000,)

Shape of x_test: (10000, 28, 28)
Shape of y_test: (10000,)
x_train shape: (60000, 28, 28, 1)
Training samples: 60000
Test samples: 10000
Epoch 4/6

Epoch 4: saving model to ./data/00001-cont-train/model-04-0.07.hdf5

Epoch 00004: saving optimizaer to ./data/00001-cont-train/model-04-0.07.pkl
469/469 - 3s - loss: 0.1272 - accuracy: 0.9634 - val_loss: 0.0692 - val_accuracy: 0.
9788 - lr: 7.5000e-05 - 3s/epoch - 6ms/step
Epoch 5/6

Epoch 5: saving model to ./data/00001-cont-train/model-05-0.06.hdf5

Epoch 00005: saving optimizaer to ./data/00001-cont-train/model-05-0.06.pkl
469/469 - 2s - loss: 0.1099 - accuracy: 0.9677 - val_loss: 0.0612 - val_accuracy: 0.
9818 - lr: 5.6250e-05 - 2s/epoch - 5ms/step
Epoch 6/6

Epoch 6: saving model to ./data/00001-cont-train/model-06-0.06.hdf5

Epoch 00006: saving optimizaer to ./data/00001-cont-train/model-06-0.06.pkl
469/469 - 2s - loss: 0.0990 - accuracy: 0.9711 - val_loss: 0.0561 - val_accuracy: 0.
9827 - lr: 5.6250e-05 - 2s/epoch - 5ms/step
Test loss: 0.05610647052526474
Test accuracy: 0.982699990272522
Elapsed time: 0:00:11.72
```